



UNIVERSIDADE CATÓLICA DE SANTOS

Ciência da Computação

Iniciação Científica

Quantização e Pruning em Operações de Transformers

Validação Conceitual e Benchmarks Controlados em GPU

Lucas Cerqueira Galvão

Santos – SP, setembro de 2026

Sumário

1	Resumo	1
2	Introdução	2
2.1	Problema	2
2.2	Objetivo	2
3	Fundamentação	3
3.1	Self-attention	3
3.2	Pruning	3
3.3	Quantização	3
4	Metodologia	4
4.1	Perguntas de pesquisa	4
4.2	Contrato de evidencia	4
4.3	Validação conceitual	4
4.4	Benchmark sintetico	5
4.5	Benchmark fisico	5
5	Resultados	5
5.1	Equivalencia conceitual	5
5.2	Fidelidade no benchmark sintetico	6
5.3	Latencia no benchmark sintetico	6
5.4	Resultados fisicos	7
5.5	Modelos OPT pre-treinados	9
5.6	Extensao de escala com OPT-2.7B e OPT-6.7B	10
5.7	Kernels confirmados	12
6	Discussão	12
6.1	Respostas parciais as perguntas de pesquisa	13
6.2	Ameaças a validade	13
7	Proximos passos	14
8	Conclusão	14

1 Resumo

Este trabalho investiga os efeitos de quantização INT8 e pruning por magnitude sobre operações centrais de arquiteturas Transformer durante inferência. O estudo separa quatro níveis de evidência: validação conceitual, equivalência numérica entre implementações, benchmark sintético robusto e validação de caminhos físicos de hardware. Foram aprovadas 18 comparações conceituais entre implementação manual, NumPy, PyTorch, PyTorch SDPA e Keras/TensorFlow. Em seguida, foram executados 2.340 registros sintéticos e 117.000 amostras individuais de latência para projeção densa e self-attention multi-head. A bateria física, executada em uma RTX 4070 Ti SUPER com Docker, PyTorch 2.11, CUDA 12.8, TorchAO e cuSPARSELt, produziu 1.650 registros e 82.500 medições. Os resultados mostram que INT8 fake preservou melhor a saída que pruning no recorte sintético; já no recorte físico, INT8 e 2:4 reduziram armazenamento e memória, mas não reduziram latência nas operações isoladas. A avaliação em OPT-125M, OPT-350M e OPT-1.3B gerou 45 registros de qualidade, 2.880 janelas de perplexidade, 900 métricas de prompts e 1.296 registros de desempenho. Nos modelos pré-treinados, INT8 preservou melhor qualidade que pruning; prefill e TTFT não sustentaram speedup pelos critérios completos, e decode permaneceu limitado por interação entre CUDA Graphs, KV cache e `torch.compile`. Uma extensão exploratória com OPT-2.7B e OPT-6.7B mostrou que a escala pode favorecer alguns caminhos de prefill, mas também aumentou o risco de degradação de qualidade quando a técnica foi aplicada aos blocos Transformer inteiros.

Palavras-chave: Transformer; self-attention; quantização; pruning; benchmark; inferência.

2 Introdução

Arquiteturas Transformer se tornaram a base de grande parte dos sistemas modernos de inteligência artificial, especialmente em modelos de linguagem e tarefas sequenciais. O mecanismo de atenção introduzido por Vaswani *et al.* (2017) substituiu recorrência explícita por operações matriciais altamente paralelizáveis, mas também concentrou custo em projeções lineares densas, produto escalar entre consultas e chaves, normalização softmax e combinação ponderada dos valores.

Esse custo é particularmente relevante durante inferência, quando o objetivo não é treinar o modelo, mas executar previsões de forma eficiente e reproduzível. Técnicas como pruning e quantização prometem reduzir custo, memória e armazenamento. O problema é que nem toda redução matemática ou lógica se transforma automaticamente em ganho físico: zeros em uma matriz densa podem continuar sendo multiplicados, e tensores dequantizados para ponto flutuante podem preservar fidelidade sem executar aritmética inteira (Han *et al.*, 2015; Jacob *et al.*, 2018; Dettmers *et al.*, 2022).

Esta iniciação científica parte dessa distinção. O objetivo não é somente comparar resultados numéricos, mas classificar que tipo de evidência cada experimento realmente sustenta. Assim, uma validação conceitual pode mostrar que duas implementações calculam a mesma operação; um benchmark sintético pode mostrar robustez entre seeds e formatos de entrada; e um experimento físico precisa confirmar representação, kernel e ambiente antes de sustentar uma alegação de desempenho.

2.1 Problema

O problema investigado é: em operações centrais de Transformers, quais compromissos entre fidelidade numérica, armazenamento, memória e latência são observados ao aplicar pruning e quantização em inferência?

Esse problema foi delimitado em dois objetos experimentais:

- **projeção densa**, usada como controle das operações lineares Q, K, V e O;
- **self-attention multi-head**, composta por projeções Q, K, V, separação em cabeças, scaled dot-product attention, recombinação e projeção de saída.

2.2 Objetivo

O objetivo geral é avaliar, em cenários controlados de inferência, os efeitos de pruning e quantização sobre operações de Transformers. Os objetivos específicos são:

- validar conceitualmente a implementação de self-attention;
- comparar implementações independentes e bibliotecas consolidadas;

- medir fidelidade numerica entre baseline, pruning e quantização;
- verificar se os efeitos persistem entre seeds, perfis e distribuições;
- distinguir evidencias logicas de evidencias fisicas de hardware;
- repetir a analise em modelos OPT pre-treinados.

3 Fundamentação

3.1 Self-attention

Na scaled dot-product attention, uma entrada é projetada em consultas (Q), chaves (K) e valores (V). A matriz QK^T mede similaridade entre posições, e a divisão por $\sqrt{d_k}$ estabiliza a escala dos logits antes da softmax. A saída é calculada pela combinação ponderada de V . Em multi-head attention, esse procedimento é executado em cabeças paralelas, concatenado e projetado novamente.

No estudo, a atenção multi-head é tratada como objeto principal porque representa o bloco conceitual efetivamente usado em Transformers. A projeção densa permanece no experimento por ser o componente linear compartilhado por Q , K , V e O .

3.2 Pruning

Pruning remove ou zera parâmetros considerados menos relevantes. O pruning por magnitude usa o valor absoluto dos pesos como critério simples: pesos com menor magnitude são removidos primeiro (Han *et al.*, 2015). Neste trabalho, o pruning denso é usado nos cenários conceituais e sintéticos para observar erro numérico. O pruning semiestruturado 2:4 é usado nos cenários físicos porque pode acionar kernels específicos em hardware NVIDIA via cuSPARSElt (NVIDIA Corporation, 2026).

3.3 Quantização

Quantização reduz a precisão numérica dos tensores. A forma fake INT8 simétrica por linha quantiza e dequantiza os pesos, preservando execução em ponto flutuante para medir fidelidade. Já os caminhos físicos diferenciam INT8 weight-only, que evidencia armazenamento compacto de pesos, e INT8 dinâmico, que também quantiza ativações e pode confirmar computação inteira no profiler (Jacob *et al.*, 2018; PyTorch Foundation, 2026).

4 Metodologia

4.1 Perguntas de pesquisa

O protocolo experimental foi organizado pelas seguintes perguntas:

RQ1 Como pruning e quantização alteram a fidelidade da saída?

RQ2 As técnicas reduzem armazenamento ou memória?

RQ3 As técnicas reduzem latência em operações isoladas?

RQ4 O tipo de evidência permite distinguir fidelidade, representação e kernel físico?

RQ5 As tendências persistem entre seeds, distribuições e perfis?

RQ6 Como a degradação varia com sparsity de 10%, 25%, 50% e 75%?

RQ7 As tendências aparecem também em modelos OPT pre-treinados?

RQ8 Quais técnicas reduzem armazenamento, memória ou latência usando caminhos físicos compatíveis?

4.2 Contrato de evidência

Uma tendência foi considerada sustentada quando ocorreu em pelo menos 80% dos casos pareados e o intervalo bootstrap de 95% não cruzou o efeito nulo. Foi classificada como mista quando dependeu de seed, perfil ou operação, e como contradita quando a direção oposta foi predominante e consistente. Ganho físico relevante exigiu speedup mediano de pelo menos 1,05x, intervalo de 95% inteiramente acima de 1,0x e confirmação do kernel correto.

4.3 Validação conceitual

Antes dos benchmarks, a implementação foi comparada com referências independentes. Foram avaliadas 18 comparações envolvendo NumPy manual, PyTorch manual, PyTorch SDPA e Keras/TensorFlow (Paszke *et al.*, 2019; Abadi *et al.*, 2016). Os casos incluem a atenção sem mascaramento, mascaramento aditivo, projeções Q/K/V/O e multi-head attention com $H \in \{1, 4, 8\}$. Todas as comparações usaram entradas e pesos compartilhados, com tolerâncias `atol=1e-6` e `rtol=1e-5`.

4.4 Benchmark sintetico

O benchmark sintetico v3 variou seeds, formas de entrada, numero de cabeças e distribuições. Foram usados 13 perfis de entrada, cinco seeds, seis cenarios e tres repetições para duas operações, totalizando 2.340 registros agregados e 117.000 amostras individuais de latencia.

Tabela 1: Estrutura do benchmark sintetico v3

Dimensao	Configuração
Operações	projeção densa e self-attention multi-head
Perfis	13 combinações de batch, sequencia, dimensão e cabeças
Seeds	11, 23, 42, 101 e 2026
Cenarios	FP32, INT8 fake, pruning 10%, 25%, 50% e 75%
Medição	20 warm-ups e 50 medições por registro

4.5 Benchmark fisico

O benchmark fisico foi executado em Linux via Docker Desktop com backend WSL2, imagem PyTorch 2.11, CUDA 12.8, TorchAO e cuSPARSELt. A GPU alvo foi uma NVIDIA RTX 4070 Ti SUPER com compute capability 8.9. Antes das medições, um probe confirmou CUDA, Triton, conversão 2:4, forward finito e operadores registrados no profiler (Docker Inc., 2026; NVIDIA Corporation, 2026; PyTorch Foundation, 2026).

Os cenarios fisicos foram:

- baseline BF16 compilado;
- INT8 weight-only por linha;
- INT8 dinamico de ativações e pesos;
- baseline FP16 compilado;
- pruning semiestruturado 2:4 em FP16.

5 Resultados

5.1 Equivalencia conceitual

As 18 comparações foram aprovadas. O maior erro absoluto registrado foi $4,76837158 \times 10^{-7}$, valor compativel com arredondamento de ponto flutuante. Assim, a implementação manual e as bibliotecas avaliadas executam a mesma construção computacional dentro das tolerancias definidas. Essa conclusão autoriza os benchmarks seguintes a compararem transformações de pruning e quantização sobre um objeto experimental conceitualmente validado.

5.2 Fidelidade no benchmark sintético

No recorte sintético, a quantização fake por linha apresentou MSE menor que pruning em 100% dos casos pareados. A degradação do pruning cresceu monotonicamente com a sparsity, como esperado.

Tabela 2: MSE medio por cenario no benchmark sintetico v3

Cenário	Projeção densa	Multi-head attention
INT8 fake por linha	$5,854 \times 10^{-5}$	$1,349 \times 10^{-5}$
Pruning 10%	$5,676 \times 10^{-4}$	$1,285 \times 10^{-4}$
Pruning 25%	$8,984 \times 10^{-3}$	$2,003 \times 10^{-3}$
Pruning 50%	$7,669 \times 10^{-2}$	$1,425 \times 10^{-2}$
Pruning 75%	$2,971 \times 10^{-1}$	$3,394 \times 10^{-2}$

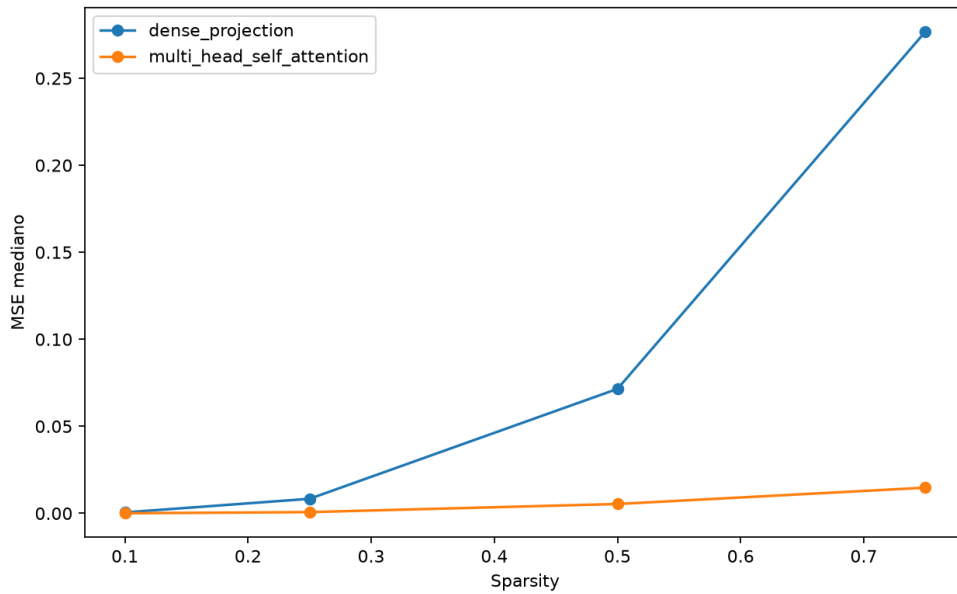


Figura 1: Degradação numerica por nivel de sparsity no benchmark sintetico.

5.3 Latencia no benchmark sintético

Os tempos do benchmark sintético não sustentam conclusão de aceleração física. As matrizes permanecem densas, a quantização fake retorna a FP32, e nenhum kernel INT8 ou sparse é usado. Por isso, a bateria sintética foi interpretada como evidência de fidelidade e robustez numérica, não como prova de desempenho em hardware.

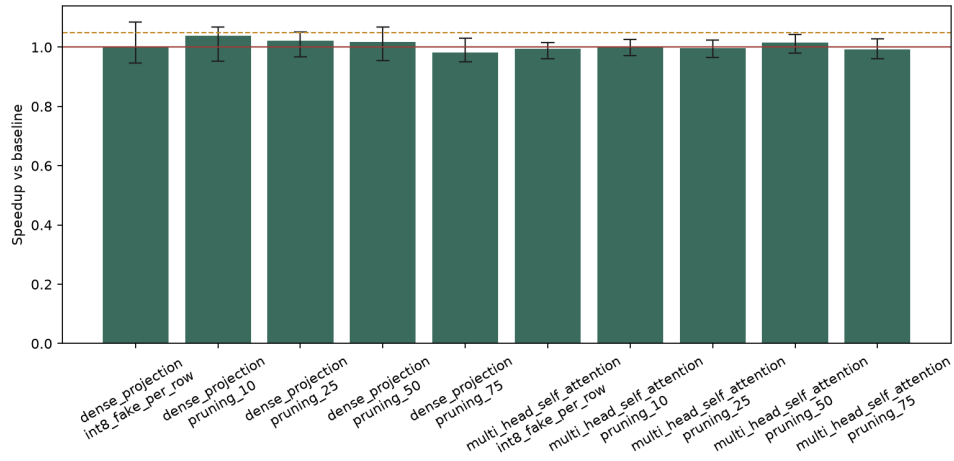


Figura 2: Speedup sintético com intervalos bootstrap de 95%; sem confirmação de kernel físico.

5.4 Resultados físicos

Na bateria física, todos os candidatos reduziram armazenamento e memória de pico mediana. No entanto, os seis grupos de desempenho tiveram speedup mediano abaixo de 1,0x, com intervalos bootstrap inteiramente abaixo do baseline. Logo, para estes shapes, esta GPU e esta implementação, a hipótese de ganho de latência foi contradita.

Tabela 3: Resumo físico por operação e cenário

Operação	Cenário	Speedup mediano	Memória mediana	Armazenamento
Projeção densa	INT8 dinâmico	0,808	0,801	0,504
Projeção densa	INT8 weight-only	0,136	0,801	0,504
Projeção densa	pruning 2:4	0,823	0,825	0,564
Multi-head attention	INT8 dinâmico	0,650	0,639	0,503
Multi-head attention	INT8 weight-only	0,097	0,639	0,503
Multi-head attention	pruning 2:4	0,645	0,728	0,562

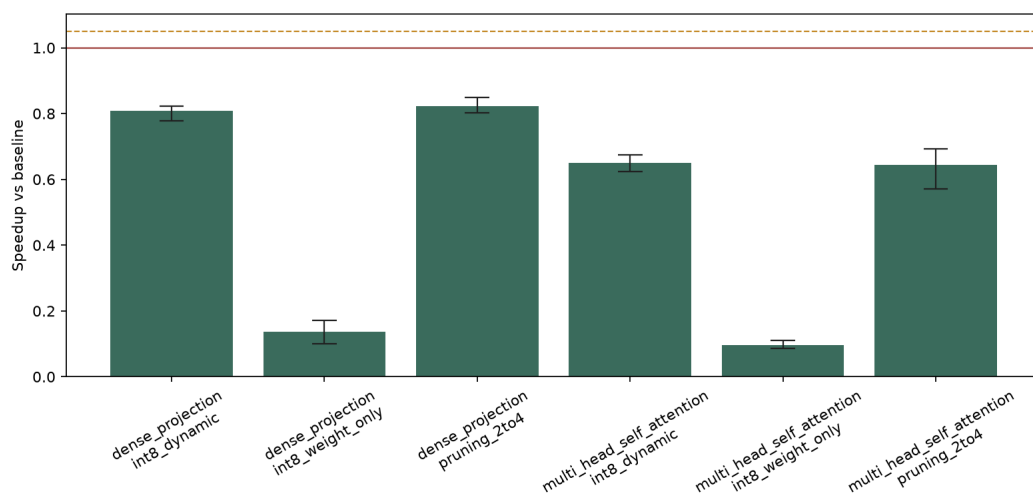


Figura 3: Speedup fisico com intervalos bootstrap de 95%; todos os grupos ficaram abaixo do baseline.

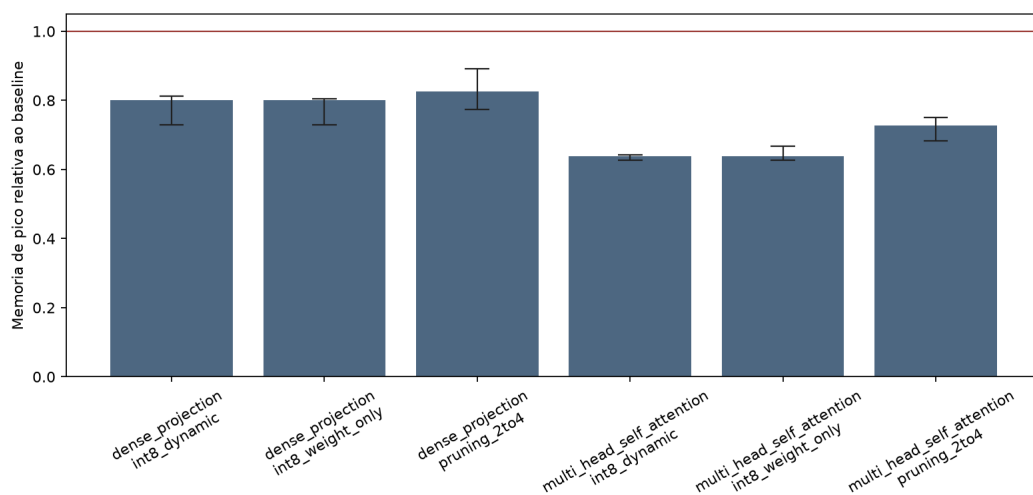


Figura 4: Memoria de pico relativa ao baseline nos cenarios fisicos.

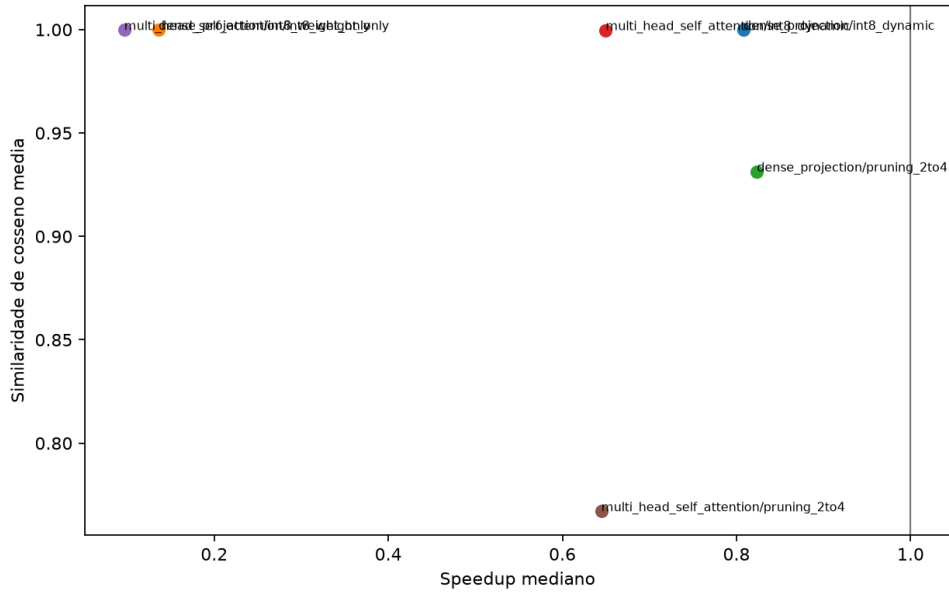


Figura 5: Fronteira qualidade-desempenho para os cenários físicos avaliados.

5.5 Modelos OPT pre-treinados

Foram avaliados OPT-125M, OPT-350M e OPT-1.3B (Zhang *et al.*, 2022), sempre partindo das revisões originais e sem treinamento. A coleta usou 64 janelas de 512 tokens do WikiText-2 e 20 prompts locais para comparar logits e geração. A bateria produziu 45 registros de qualidade, 2.880 janelas, 900 métricas de prompts, 1.296 registros de desempenho e 10.080 amostras individuais de timing.

Tabela 4: Perplexidade dos modelos OPT em cenários representativos

Modelo	Baseline BF16	INT8 fake atencao	Pruning 50% atencao
OPT-125M	42,371	42,313	65,796
OPT-350M	33,345	33,348	50,668
OPT-1.3B	21,394	21,384	28,644

Vinte e cinco das 45 variantes ficaram dentro dos limites operacionais. As quantizações INT8 fake e weight-only foram estáveis nos três modelos. O INT8 dinâmico foi aceitável em OPT-125M e OPT-350M, mas excedeu o limite em OPT-1.3B. O pruning por magnitude em atenção foi aceitável apenas em 10%; a partir de 25%, a perplexidade já excedeu o limite de 5%.

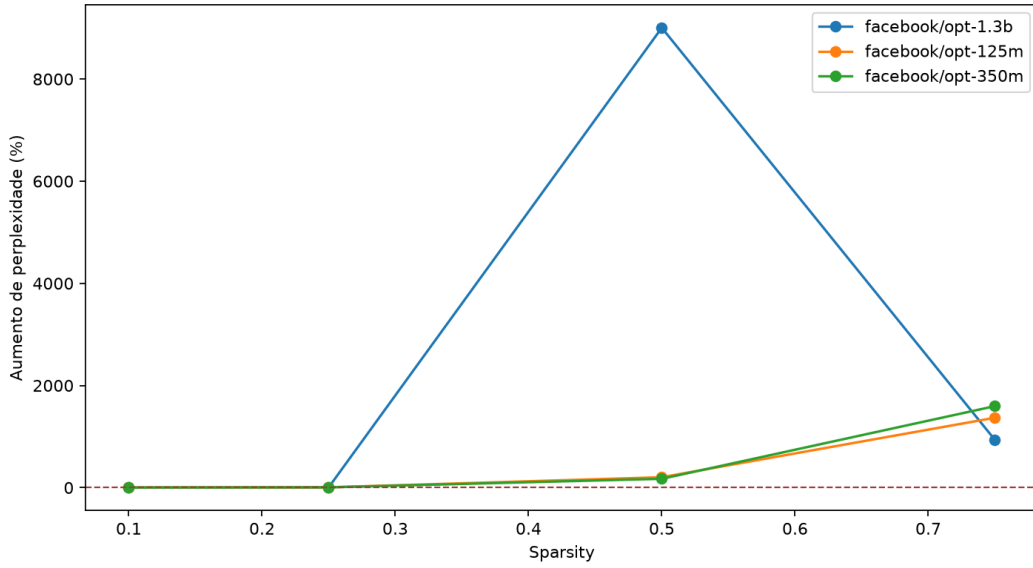


Figura 6: Perplexidade por sparsity e modelo na avaliacao OPT.

Em desempenho, prefill e TTFT foram medidos com registros completos. Nenhum cenario cumpriu simultaneamente speedup mediano de pelo menos 1,05x, intervalo de 95% acima de 1,0x e kernel confirmado. O caso mais favoravel foi `blocks_int8_dynamic` em OPT-1.3B, mas o intervalo de confianca ainda cruzou o efeito nulo. Todos os casos planejados de decode foram preservados como falha tecnica ou incompatibilidade: o problema observado foi a interacao entre `torch.compile`, CUDA Graphs e atualizacao de KV cache.

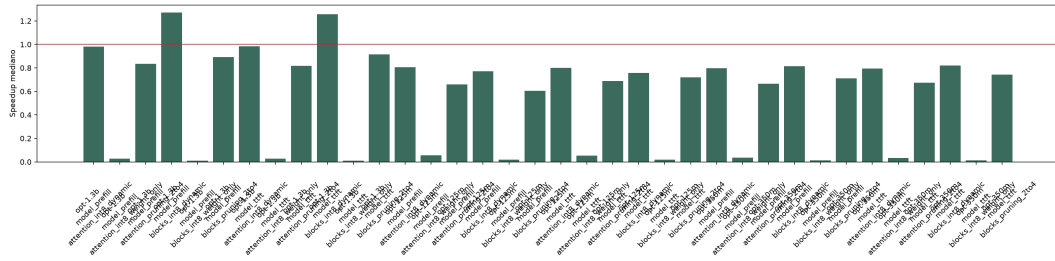


Tabela 5: Qualidade e desempenho em modelos OPT maiores

Modelo	Cenário	Perplexidade	Aumento vs baseline	Speedup prefill	IC95 prefill
OPT-2.7B	baseline BF16	18,178	0,0%	–	–
OPT-2.7B	atencao INT8 dinamico	18,955	4,27%	1,024	[0,844; 1,134]
OPT-2.7B	blocos INT8 dinamico	18,981	4,42%	1,305	[0,907; 1,588]
OPT-2.7B	atencao 2:4	27,670	52,22%	0,873	[0,798; 0,907]
OPT-2.7B	blocos 2:4	519,074	2755,50%	1,061	[0,975; 1,078]
OPT-6.7B	baseline BF16	15,346	0,0%	–	–
OPT-6.7B	atencao INT8 dinamico	16,659	8,55%	1,169	[1,009; 1,249]
OPT-6.7B	blocos INT8 dinamico	40,675	165,05%	1,756	[1,171; 2,137]
OPT-6.7B	atencao 2:4	21,311	38,87%	1,030	[0,974; 1,111]
OPT-6.7B	blocos 2:4	168,097	995,38%	1,073	[1,017; 1,228]

O resultado muda a leitura da hipótese de escala. Em OPT-6.7B, o INT8 dinâmico aplicado aos blocos atingiu speedup mediano de 1,756 em prefill e 1,744 em TTFT, com intervalos de 95% acima de 1,0. Contudo, a perplexidade subiu de 15,346 para 40,675, aumento de 165%. Portanto, há ganho físico, mas ele não é aceitável pelo critério operacional de qualidade. O mesmo vale para 2:4 aplicado aos blocos: o speedup de prefill fica acima de 1,05 com intervalo acima de 1,0, mas a perplexidade sobe para 168,097. A conclusão correta é que a escala pode tornar alguns kernels mais rápidos, mas não garante uma configuração útil se a qualidade colapsar.

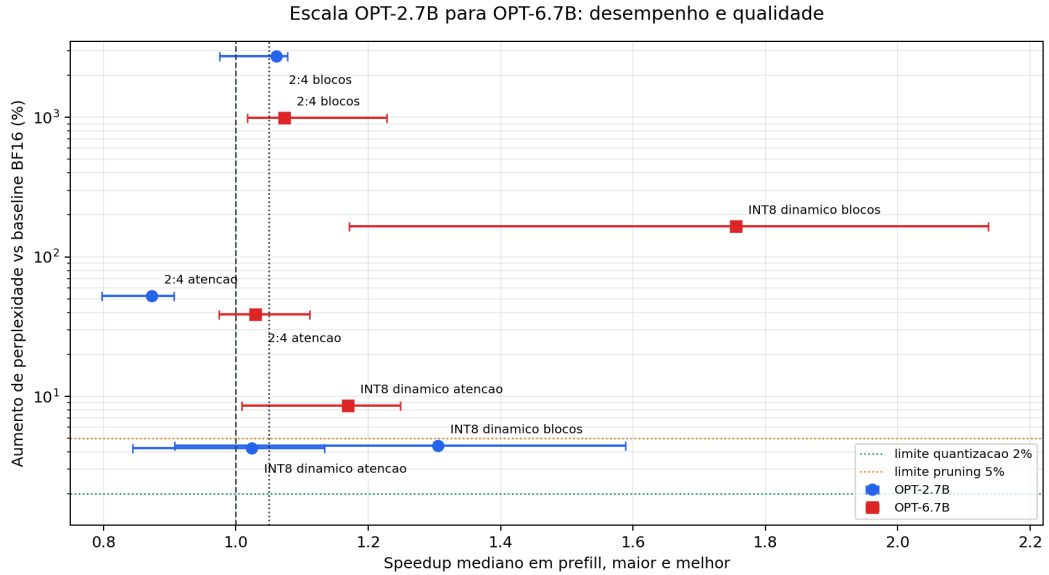


Figura 8: Comparação entre OPT-2.7B e OPT-6.7B no compromisso entre speedup de prefill e aumento de perplexidade.

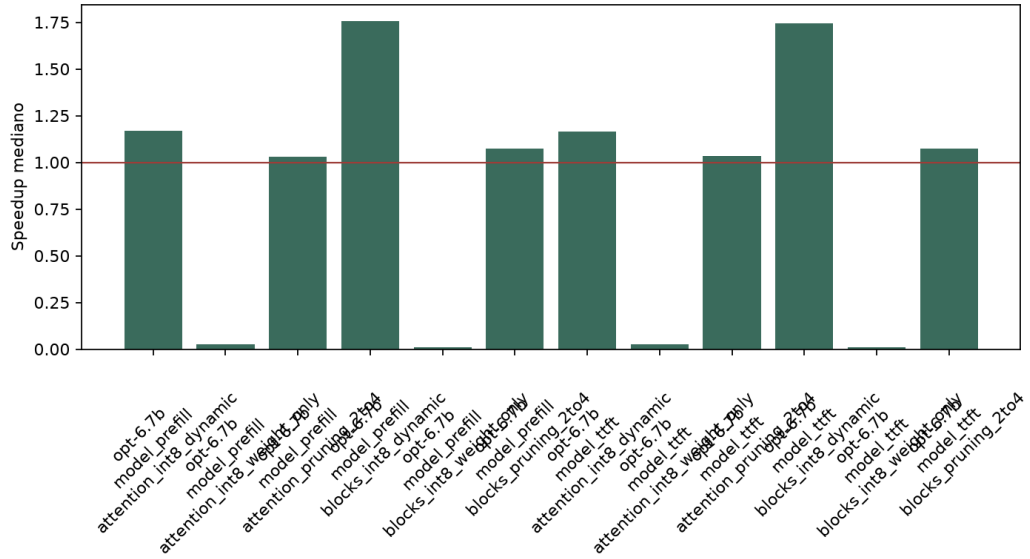


Figura 9: Speedup em prefill e TTFT na extensão OPT-6.7B.

5.7 Kernels confirmados

O profiler registrou caminhos distintos para cada tecnica fisica. O INT8 weight-only acionou kernel de pesos empacotados, sendo classificado como evidencia de armazenamento compacto. O INT8 dinamico registrou GEMM S8 via CUTLASS, sustentando computação inteira. O pruning 2:4 registrou sparse GEMM por cuSPARSELt, sustentando representação semiestruturada.

Tabela 6: Classificação dos caminhos fisicos

Cenario	Evidencia confirmada	Conclusão permitida
INT8 fake	dequantização para FP32	fidelidade numerica
INT8 weight-only	pesos empacotados INT8	armazenamento compacto
INT8 dinamico	GEMM S8 no profiler	computação inteira
Pruning 2:4	sparse GEMM cuSPARSELt	caminho sparse fisico

6 Discussão

Os resultados reforçam a necessidade de separar as camadas de evidencia. No plano conceitual, a self-attention implementada manualmente e as versões de frameworks consolidados chegaram ao mesmo resultado. No plano numerico, INT8 fake preservou melhor a saída que pruning denso. No plano fisico, tecnicas com representação real reduziram armazenamento e memoria, mas o custo de conversão, empacotamento, kernel e shape impediu ganho de latencia nas operações isoladas. Nos modelos OPT, a mesma tendencia de qualidade foi observada parcialmente: INT8 preservou perplexidade melhor que pruning, enquanto pruning degradou

rapidamente conforme a sparsity aumentou. A extensao de escala mostrou um resultado mais nuancado: em OPT-6.7B, INT8 dinamico e pruning 2:4 aplicados aos blocos tiveram speedup de prefill com intervalo acima de 1,0, mas ambos ficaram fora dos limites de qualidade.

Isso nao significa que INT8 ou pruning 2:4 sejam tecnicas ineficazes de modo geral. A conclusao correta e mais estreita: na bateria principal, nao houve speedup sustentado e aceitavel pelos criterios completos; na extensao OPT-6.7B, houve speedup fisico em alguns caminhos, mas acompanhado de degradacao de perplexidade. Essa leitura evita extrapolar benchmarks sinteticos, zeros logicos ou speedup isolado sem verificar qualidade.

6.1 Respostas parciais as perguntas de pesquisa

Pergunta	Resposta ate esta versao
RQ1	INT8 fake preservou melhor a saida que pruning em 100% dos casos sinteticos pareados.
RQ2	INT8 fisico reduziu armazenamento para cerca de 50% e pruning 2:4 para cerca de 56%; a memoria de pico tambem caiu.
RQ3	Nenhum cenario fisico sustentou reducao de latencia; nos OPT, prefill e TTFT tambem nao cumpriram todos os criterios.
RQ4	A separacao entre fidelidade, armazenamento, computacao inteira e sparse fisico foi confirmada.
RQ5	As tendencias numericas persistiram entre cinco seeds, perfis e distribuicoes.
RQ6	A degradacao aumentou conforme a sparsity passou de 10% para 75%.
RQ7	Nos OPT, INT8 preservou melhor qualidade que pruning; a degradacao de pruning cresceu com sparsity; na extensao 6.7B o speedup apareceu, mas nao permaneceu aceitavel quando a qualidade foi considerada.
RQ8	INT8 e 2:4 reduziram armazenamento e memoria com caminhos compatíveis; apenas na extensao 6.7B alguns caminhos tiveram speedup fisico, sem cumprir simultaneamente os limites de qualidade.

6.2 Ameaças a validade

As principais ameacas sao a limitacao da medicao de decode com KV cache no compilador do PyTorch; a possibilidade de resultados diferentes em outros shapes, GPUs ou bibliotecas; e a diferenca entre operacoes isoladas, prefill, TTFT e execucao ponta a ponta de um modelo de linguagem. Alem disso, medicoes de GPU podem sofrer interferencia de processos concorrentes, clocks dinamicos e temperatura. Por isso, os runners exigem GPU ociosa antes da coleta final.

7 Proximos passos

A etapa seguinte é decidir como tratar a medição de decode. Uma alternativa é reexecutar apenas esse recorte com marcação explícita de passos de CUDA Graph ou sem o modo `reduce-overhead`; outra é declarar decode fora do escopo desta versão e concentrar a conclusão em qualidade, prefill, TTFT, memória e armazenamento. Também resta ajustar o texto para o formato específico de submissão escolhido.

8 Conclusão

O estudo validou conceitualmente a implementação de self-attention e consolidou três conjuntos experimentais robustos. A evidência sintética mostra que INT8 fake preserva melhor a saída que pruning por magnitude, enquanto a evidência física mostra redução de armazenamento e memória para INT8 e 2:4, mas contradiz ganho de latência nas operações isoladas avaliadas.

A contribuição principal é metodológica e experimental: o trabalho evita misturar equivalência matemática, representação compacta e aceleração física como se fossem a mesma evidência. A conclusão final mostra que os efeitos observados nos benchmarks controlados aparecem parcialmente em modelos pre-treinados: INT8 preserva qualidade melhor que pruning, pruning degrada com sparsity, e redução de representação ou memória não garante latência menor. A extensão com OPT-6.7B acrescenta que speedup pode surgir em escala maior, mas esse ganho só é cientificamente útil quando a qualidade também permanece dentro dos limites definidos.

Referências

ABADI, Martín *et al.* TensorFlow: A System for Large-Scale Machine Learning. *In*: 12TH USENIX Symposium on Operating Systems Design and Implementation. [S. l.: s. n.], 2016.

DETTMERS, Tim *et al.* LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. *In*: ADVANCES in Neural Information Processing Systems. [S. l.: s. n.], 2022.

DOCKER INC. **Docker Desktop GPU Support Documentation**. [S. l.: s. n.], 2026.
<https://docs.docker.com/desktop/features/gpu/>. Acesso em: set. 2026.

HAN, Song *et al.* Learning both Weights and Connections for Efficient Neural Networks. *In*: ADVANCES in Neural Information Processing Systems. [S. l.: s. n.], 2015.

JACOB, Benoit *et al.* Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. *In*: PROCEEDINGS of the IEEE Conference on Computer Vision and Pattern Recognition. [S. l.: s. n.], 2018.

NVIDIA CORPORATION. **cuSPARSELt Release Notes**. [S. l.: s. n.], 2026.
https://docs.nvidia.com/cuda/cusparselt/release_notes.html.
Acesso em: set. 2026.

PASZKE, Adam *et al.* PyTorch: An Imperative Style, High-Performance Deep Learning Library. *In: ADVANCES in Neural Information Processing Systems*. [S. l.: s. n.], 2019.

PYTORCH FOUNDATION. **TorchAO Quantized Inference Documentation**. [S. l.: s. n.], 2026.
<https://docs.pytorch.org/ao/stable/workflows/inference.html>.
Acesso em: set. 2026.

VASWANI, Ashish *et al.* Attention Is All You Need. *In: ADVANCES in Neural Information Processing Systems*. [S. l.: s. n.], 2017.

ZHANG, Susan *et al.* OPT: Open Pre-trained Transformer Language Models. **arXiv preprint arXiv:2205.01068**, 2022.